

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)
Department of Computer Science and Engineering
CO2 ASSESSMENT TOOL 2 – Architecture Design Assignment

Course Code:	CSA10 / CSA1063	Course Title:	Software Engineering
CO Assessed:	CO2	Assessment:	Assessment Tool 2 – Architecture Design Assignment
Weightage:	20%	Total Marks:	25
CO2 Statement:	<i>Perform detailed requirements analysis, design scalable architectures, and prioritize features with a focus on cross-disciplinary skills</i>		
Student Name:	Joanathan Packia Singh	Reg. No.:	192472229
Date:	06/08/2026	Duration:	60 minutes

Code of Conduct

I Joanathan Packia Singh/192472229 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: Joanathan

Annexure A – Architecture Design Assignment

Instructions to Students:

Each student will be assigned an **individual software project problem statement**. Based on the assigned problem, analyze the system requirements and design an appropriate software architecture by applying software engineering principles. Your submission should demonstrate your ability to select, justify, and evaluate a suitable architectural style.

Question

Based on your **assigned problem statement**, perform an **Architecture Design Analysis** and prepare an architecture design report by answering the following questions:

1. Analyze System Requirements

- Study the given problem statement and identify the system objectives.
- Analyze the functional and non-functional requirements that influence the software architecture.
- Identify key quality attributes such as scalability, maintainability, reliability, availability, performance, and security.

2. Select a Suitable Software Architecture

- Select an appropriate architectural style for the proposed system (e.g., Layered Architecture, Client-Server, MVC, Microservices, Event-Driven, Service-Oriented Architecture, Serverless, or Hybrid Architecture).
- Justify why the selected architecture is the most suitable for the given problem.

3. Draw the Software Architecture Diagram

- Design a clear architecture diagram illustrating the overall structure of the proposed system.
- Include all major layers/components, databases, external systems, APIs, and communication mechanisms.
- Use appropriate software architecture notations and labels.

4. Explain Components and Their Interactions

- Describe the purpose and responsibilities of each architectural component.
- Explain how the components communicate and exchange data.
- Illustrate the overall workflow of the system.

5. Discuss Quality Attributes

Evaluate how the proposed architecture addresses the following aspects:

- Scalability
- Security
- Performance
- Reliability
- Maintainability
- Availability

6. Justify Architectural Decisions

- Explain the rationale behind your architectural choices.
- Discuss the trade-offs considered while selecting the architecture.
- Explain how the chosen architecture supports future enhancements, system integration, and long-term maintainability.

Rubrics for Calculation

Evaluation Criteria	Excellent (4 Marks)	Good (3 Marks)	Satisfactory (2 Marks)	Needs Improvement (1 Mark)
System Requirement Analysis (30%)	Thoroughly analyzes the problem statement, accurately identifies functional and non-functional requirements, and clearly explains quality attributes influencing the architecture.	Analyzes most requirements with minor omissions or limited explanation.	Identifies only basic requirements with partial analysis.	Requirement analysis is incomplete, inaccurate, or lacks clarity.
Selection of Software Architecture (25%)	Selects the most appropriate architectural style with strong technical justification aligned to system requirements.	Selects a suitable architecture with reasonable justification.	Architecture is partially suitable with limited justification.	Inappropriate architecture selected or justification is missing.
Architecture Diagram and Component Design (20%)	Produces a clear, complete, and well-structured architecture diagram with all major components, interfaces, and interactions accurately represented.	Diagram is mostly complete with minor omissions or notation issues.	Diagram includes only essential components and lacks sufficient detail.	Diagram is incomplete, unclear, or technically incorrect.
Component Interaction and	Clearly explains component	Explains most components and	Provides limited explanation of	Component interactions and

Quality Attribute Analysis (15%)	responsibilities, interactions, and thoroughly discusses scalability, security, performance, reliability, maintainability, and availability.	quality attributes with minor gaps.	interactions and addresses only some quality attributes.	quality attributes are poorly explained or missing.
Architectural Justification and Technical Presentation (10%)	Provides strong justification for architectural decisions, discusses trade-offs and future enhancements, and presents a well-organized, professional report.	Provides adequate justification with minor gaps; report is well organized.	Limited justification with minimal discussion of trade-offs or future scope; report needs improvement.	Justification is weak or absent; report lacks organization and technical clarity.

Criteria	Mark
System Requirement Analysis (30%)	/5
Selection of Software Architecture (25%)	/5
Architecture Diagram and Component Design (20%)	/5
Component Interaction and Quality Attribute Analysis (15%)	/5
Architectural Justification and Technical Presentation (10%)	/5
TOTAL	/25



SIMATS ENGINEERING

Saveetha Institute of Medical and Technical Sciences
Chennai-602105



CO2 ASSESSMENT TOOL 2 — ARCHITECTURE DESIGN ASSIGNMENT

Course Code:	CSA10 / CSA1063	Course Title:	Software Engineering
CO Assessed:	CO2	Assessment:	Assessment Tool 2 – Architecture Design Assignment
Weightage:	20%	Total Marks:	25
Student Name:	Joanathan Packia Singh	Reg. No.:	192472229
Date:	06 August 2026	Duration:	60 minutes
Assigned Problem Statement:	22. Fitness and Diet Tracker with Personalized Recommendations		

Code of Conduct

I Joanathan Packia Singh (Reg. No. 192472229) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: Joanathan Packia Singh

Annexure A — Software Architecture Design Report

Fitness and Diet Tracker with Personalized Recommendations

Executive Summary

This report presents a complete architecture design analysis for a Fitness and Diet Tracker with Personalized Recommendations. The report analyses functional and non-functional requirements, selects and justifies a Hybrid Layered plus Microservices architectural style, and presents seven professionally produced diagrams — architecture, component, sequence, data-flow, entity-relationship, deployment, and a quality-attribute radar chart. It concludes with a discussion of the six core software quality attributes, a justification of architectural trade-offs, a technology-stack summary, a risk assessment, a testing strategy, and a roadmap of future enhancements.

Document Structure

- Section 1 — Introduction and Industry Context
- Section 2 — System Requirements Analysis (Objectives, Functional and Non-Functional Requirements)
- Section 3 — Selection of Software Architecture and Justification
- Section 4 — Software Architecture Diagram and Component Diagram
- Section 5 — Component Description, Sequence, Data-Flow, ER, and Deployment Diagrams
- Section 6 — Discussion of Quality Attributes
- Section 7 — Architectural Justification and Trade-offs
- Section 8 — Technology Stack Summary
- Section 9 — API Design Overview
- Section 10 — Risk Assessment and Mitigation
- Section 11 — Testing Strategy
- Section 12 — Implementation Roadmap
- Section 13 — Glossary of Terms and Abbreviations
- Section 14 — Conclusion and References

1. Introduction and Industry Context

The global fitness and wellness industry has undergone rapid digital transformation driven by rising awareness of preventive healthcare, chronic-disease prevention, and lifestyle management. Consumers increasingly rely on smartphones and wearable devices to monitor physical activity, sleep quality, heart rate, and nutritional intake. Fitness centres, nutritionists, and healthcare providers require integrated digital platforms that can consolidate this data and translate it into actionable, personalised guidance.

Wearable devices such as Fitbit, Google Fit, and Apple Health generate continuous streams of biometric data that, when combined with dietary logs, provide a rich foundation for AI-driven personalisation. However, most existing fitness applications remain siloed — they either track activity or diet in isolation and rarely offer adaptive, data-driven recommendations tied to an individual's evolving health profile. The proposed Fitness and Diet Tracker addresses this gap by unifying activity tracking, nutrition logging, wearable synchronisation, and AI-based recommendation generation within a single, secure, and scalable software architecture.

This report performs a complete architecture design analysis for the system: it identifies functional and non-functional requirements, selects and justifies an appropriate architectural style, presents detailed architecture, component, sequence, data-flow, entity-relationship, and deployment diagrams produced using professional diagramming tools, discusses the six core quality attributes, and concludes with a justification of architectural trade-offs and a roadmap for future enhancement.

1.1 Assumptions and Constraints

- Users have consistent access to a smartphone with internet connectivity; offline logging is queued and synced once connectivity resumes.
- Wearable devices expose a public, rate-limited REST API (Fitbit, Google Fit, Apple Health) that the Wearable Sync Service can poll or subscribe to via webhooks.
- The system must comply with applicable data-protection regulations (e.g., GDPR/HIPAA-equivalent principles) given the sensitivity of personal health data.
- Initial deployment targets a single cloud region with future multi-region expansion considered but out of scope for this design iteration.
- AI recommendations are advisory in nature and are not a substitute for professional medical or nutritional advice.
- The development team has access to managed cloud services (AWS/GCP) rather than self-hosted infrastructure.

2. System Requirements Analysis

2.1 System Objectives

The system is designed to achieve the following objectives, derived directly from the assigned problem statement:

- Develop a personalised fitness and diet management platform for end users.
- Track daily physical activities and nutritional intake with high accuracy.
- Generate AI-based workout and diet recommendations tailored to individual goals.
- Integrate wearable device data for continuous, automated health monitoring.
- Visualise user progress through interactive dashboards and reports.
- Send timely reminders for exercise, hydration, and meals.
- Ensure secure storage and transmission of sensitive personal health information.
- Promote sustained healthy-lifestyle habits through continuous engagement.

2.2 Functional Requirements

- User registration, authentication, and profile management (age, weight, height, BMI, fitness goals).
- Daily logging of physical activities (steps, workouts, duration, calories burnt).
- Daily logging of meals and nutritional intake (calories, macronutrients, hydration).
- Automatic synchronisation of biometric data from connected wearable devices.
- AI-driven generation of personalised workout and diet recommendations.
- Interactive dashboards displaying progress trends, goal completion, and analytics.
- Configurable reminders and push notifications for meals, workouts, and hydration.
- Administrative and nutritionist access for reviewing aggregated, anonymised trends.

2.3 Non-Functional Requirements and Quality Attributes

Beyond core functionality, the architecture must satisfy the following quality attributes, which directly influence the choice of architectural style described in Section 3:

Quality Attribute	Requirement Description
Scalability	Support a growing user base and increasing volumes of wearable-generated telemetry without performance degradation.
Security	Protect sensitive health data using encryption in transit and at rest, role-based access control, and audit logging.
Performance	Deliver real-time dashboard updates and AI recommendations within acceptable latency thresholds (< 2 seconds).

Reliability	Guarantee accurate, consistent data capture and synchronisation even under intermittent network conditions.
Maintainability	Allow independent development, testing, and deployment of individual features without affecting the whole system.
Availability	Maintain high uptime (99.9%) through redundancy, auto-scaling, and fault-tolerant deployment.

2.4 Feature Prioritisation (MoSCoW Analysis)

To guide phased delivery, core features were classified using the MoSCoW technique, ensuring the highest-value capabilities are architected and delivered first.

Priority	Features
Must Have	User registration & profile, activity logging, diet logging, secure authentication, core dashboard.
Should Have	Wearable device synchronisation, AI-based recommendations, progress reports.
Could Have	Social/community challenges, gamification badges, nutritionist chat support.
Won't Have (this release)	Integration with clinical/EHR systems, insurance-provider data exchange.

3. Selection of Software Architecture

3.1 Selected Architectural Style

A Hybrid Layered + Microservices Architecture has been selected for the Fitness and Diet Tracker. The system is organised internally into classical horizontal layers — presentation, application/business logic, integration, and data — while the business-logic layer itself is decomposed into independently deployable microservices (User Service, Activity Tracking Service, Diet & Nutrition Service, AI Recommendation Engine, Wearable Sync Service, and Notification Service).

3.2 Justification

- Independent scalability: The AI Recommendation Engine and Wearable Sync Service experience unpredictable, bursty load (e.g., nightly batch recommendation generation or mass wearable sync), so isolating them as separate microservices allows independent horizontal scaling without over-provisioning the entire system.
- Fault isolation: If the Notification Service fails, core logging and tracking functionality remains unaffected — a critical reliability property for a health-monitoring application.
- Technology heterogeneity: The AI Recommendation Engine benefits from Python-based ML frameworks (TensorFlow/PyTorch) while other services can remain on lightweight Node.js/Java stacks — a microservices boundary makes this practical.
- Layered clarity: Retaining the layered structure within and around the microservices keeps cross-cutting concerns (authentication, gateway routing, caching, persistence) organised and easy to reason about for a student-scale capstone project.
- Faster iterative delivery: Each microservice can be developed, tested, and deployed on its own release cycle, which aligns with Agile sprint-based development already adopted for this capstone.

Pure microservices architecture was considered but rejected as the sole style because the operational overhead (service discovery, distributed tracing, inter-service contracts) is disproportionate for a project of this scope. A purely monolithic layered architecture was also rejected because it would not satisfy the independent-scalability and fault-isolation requirements identified in Section 2.3. The hybrid style therefore offers the best balance of simplicity and the quality attributes required.

3.3 Comparative Evaluation of Architectural Styles

The table below scores each candidate architectural style against the quality attributes prioritised for this system (1 = Poor, 5 = Excellent), confirming that the selected hybrid style achieves the strongest overall balance.

Architectural Style	Scalability	Complexity (lower=better)	Fault Isolation	Dev. Speed	Overall Fit
Monolithic Layered	2	5	1	5	3
Pure Client-Server	2	4	2	4	2
Pure Microservices	5	1	5	2	3
Event-Driven	4	2	4	3	3
Serverless	5	3	3	3	3
Hybrid Layered + Microservices (Selected)	5	3	4	4	5

4. Software Architecture Diagram

Figure 1 presents the complete system architecture, illustrating all major layers, components, databases, external systems, and communication mechanisms as required by the assignment.

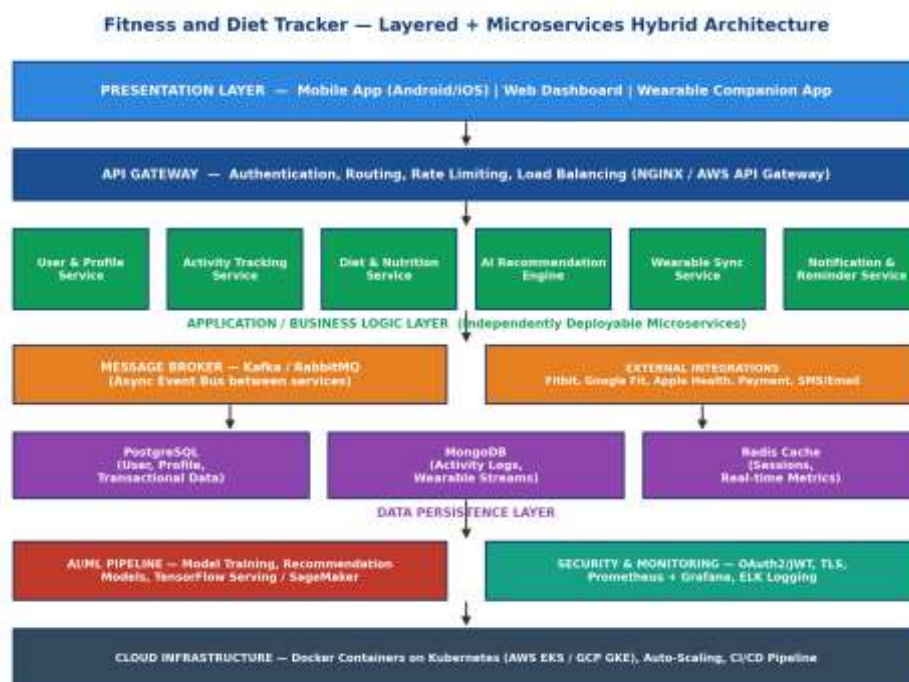
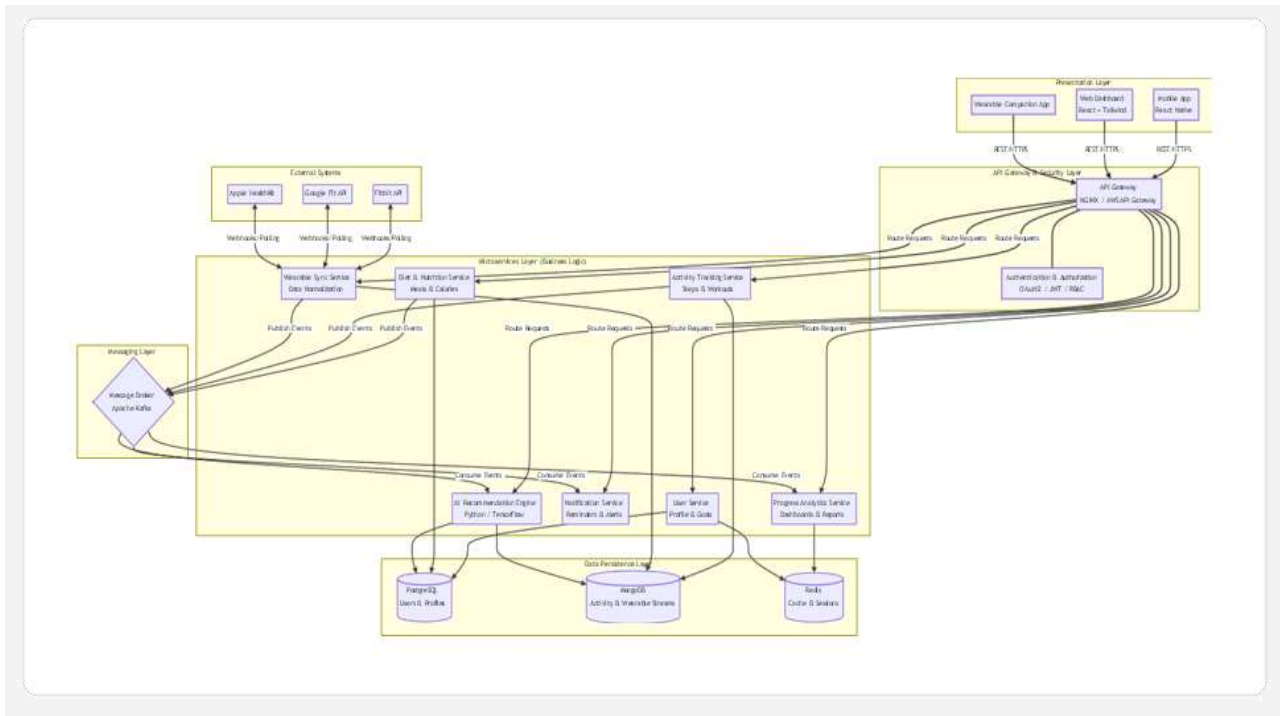


Figure 1: Layered + Microservices Hybrid Architecture of the Fitness and Diet Tracker

The Presentation Layer exposes the mobile application, web dashboard, and wearable companion app to end users. All client requests pass through a single API Gateway that performs authentication, request routing, load balancing, and rate limiting before forwarding calls to the appropriate microservice. Microservices communicate synchronously via REST/gRPC for request-response interactions and asynchronously via a Kafka/RabbitMQ message broker for event-driven workflows such as wearable-sync notifications and recommendation triggers. The Data Persistence Layer uses PostgreSQL for structured transactional data, MongoDB for high-volume activity and wearable streams, and Redis for low-latency session and real-time metric caching. The entire stack is containerised with Docker and orchestrated on Kubernetes, providing auto-scaling and a standard CI/CD pipeline for continuous delivery.

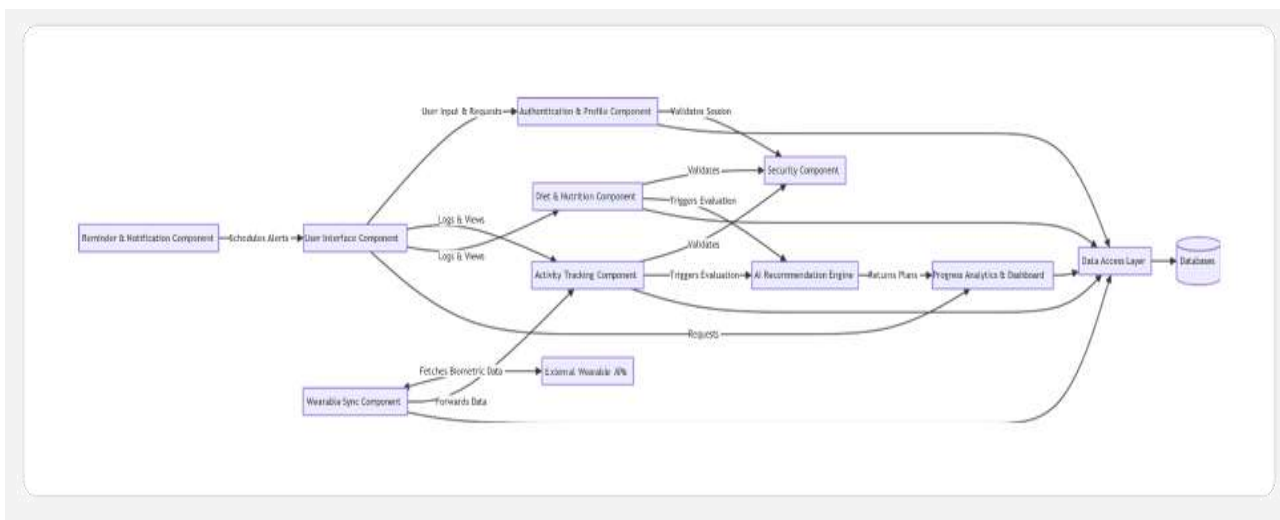
4.1 Figure 1: Layered + Microservices Hybrid Architecture Diagram

This diagram illustrates the overall system structure, showing the presentation layer, API gateway, microservices, message broker, data persistence, and external integrations.



4.2 Figure 2: Component Diagram

This diagram illustrates the overall system structure, showing the presentation layer, API gateway, microservices, message broker, data persistence, and external integrations.



4.3 Figure 3: Deployment Diagram (Cloud Infrastructure Topology)

This diagram maps the software components onto Kubernetes nodes, managed database services, and third-party integrations within a cloud environment (e.g., AWS/GCP).

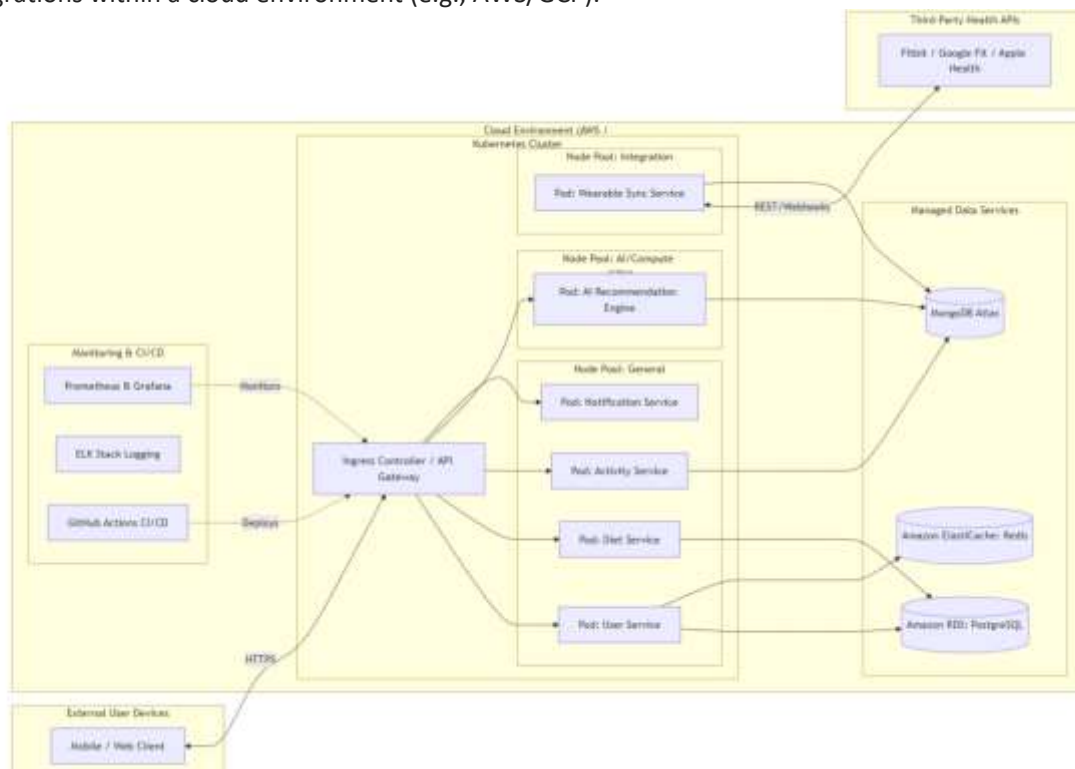


Figure 3: Deployment Diagram

4.4 Figure 4: Level-1 Data Flow Diagram (DFD)

This diagram shows how data moves between external entities, processes, and data stores across the system.

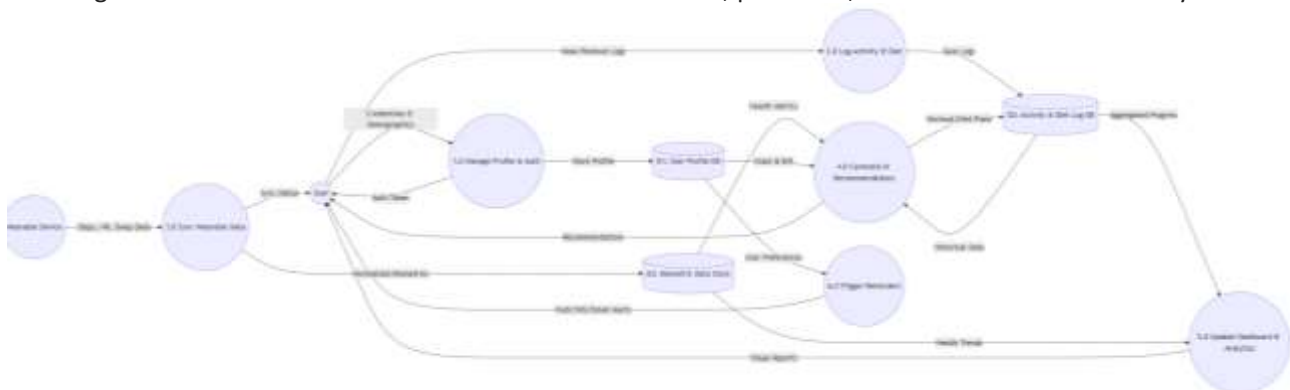


Figure 4: Level-1 Data Flow Diagram

4.5 Entity-Relationship (ER) Diagram

This diagram illustrates the underlying database design, showing principal entities, attributes, and relationships.

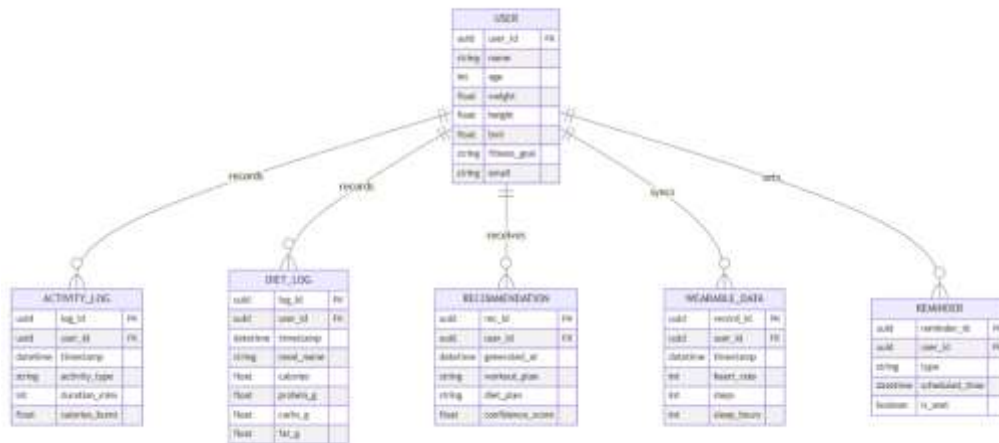


Figure 5: Entity-Relationship (ER) Diagram

5. Component Description and Interaction

5.1 Component Responsibilities

Component	Responsibility
User Interface Component	Renders mobile/web screens, captures user input, and displays dashboards and recommendations.
Authentication & Profile Component	Manages registration, login (OAuth2/JWT), and stores demographic and goal data.
Wearable Sync Component	Connects to third-party wearable APIs and normalises incoming biometric streams.
Activity Tracking Component	Records workouts, steps, and calories burnt; computes daily activity summaries.
Diet & Nutrition Component	Logs meals, calculates calories/macronutrients, and validates entries against nutrition databases.
Reminder & Notification Component	Schedules and dispatches push/SMS/email reminders for meals, workouts, and hydration.
AI Recommendation Engine	Analyses historical activity, diet, and goal data to generate personalised workout/diet plans.
Progress Analytics & Dashboard Component	Aggregates data into visual reports, trend charts, and goal-completion indicators.
Data Access Layer	Implements the repository/DAO pattern to abstract database operations from business logic.
Security Component	Provides encryption, role-based access control (RBAC), and centralised audit logging.

5.2 Sequence Diagram — Key Workflow

Figure 3 traces the runtime interaction sequence for the most representative workflow: a user logging a meal and receiving an AI-generated nutritional recommendation.

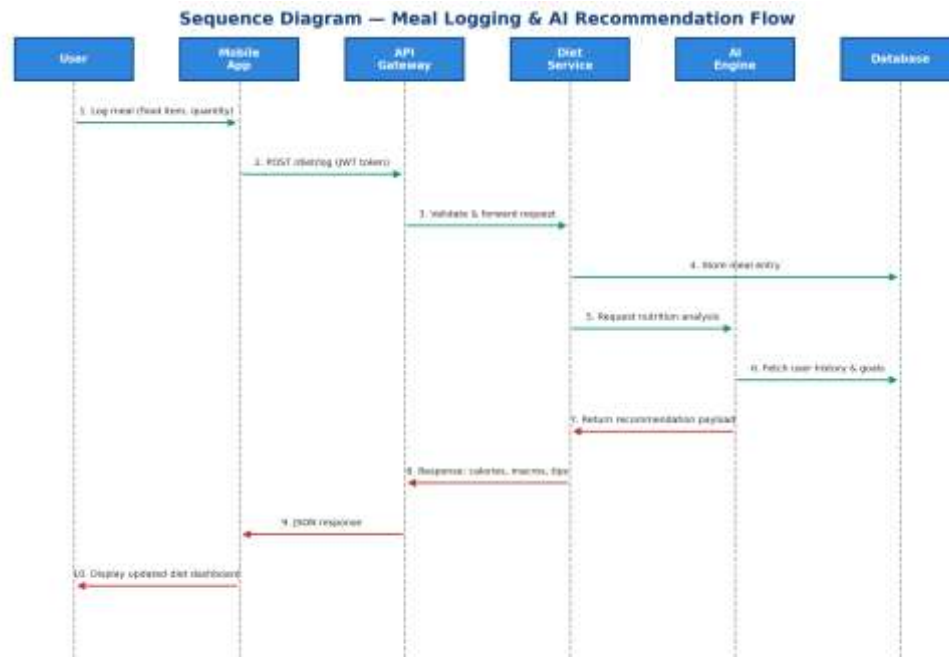


Figure 3: Sequence Diagram — Meal Logging and AI Recommendation Flow

5.3 Data Flow Diagram

Figure 4 presents the Level-1 Data Flow Diagram (DFD), showing how data moves between external entities, processes, and data stores across the system.

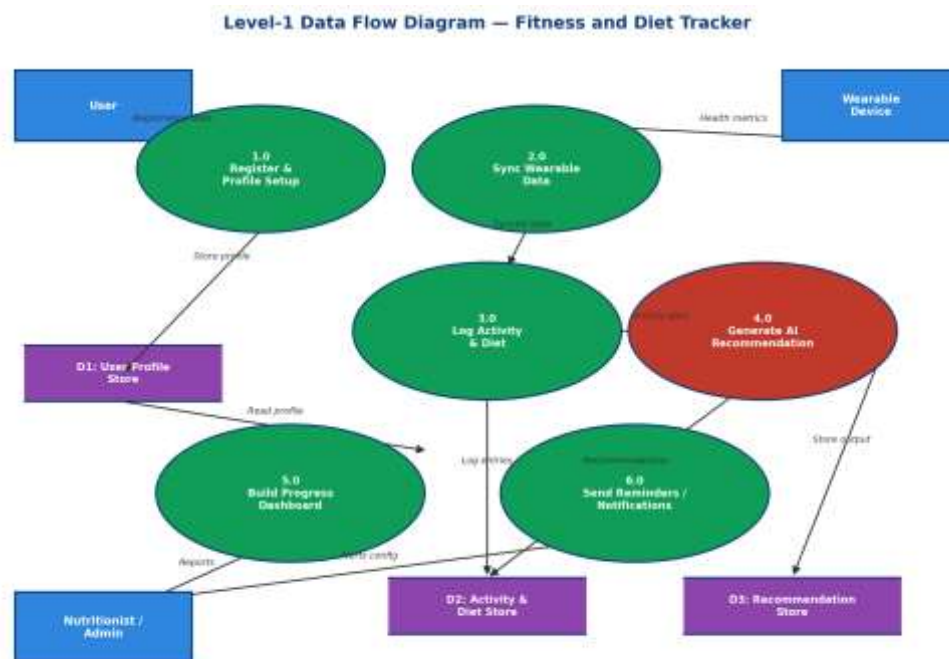


Figure 4: Level-1 Data Flow Diagram of the Fitness and Diet Tracker

5.4 Entity Relationship Diagram

Figure 5 illustrates the underlying database design, showing the principal entities, their attributes, and relationships that support the components described above.

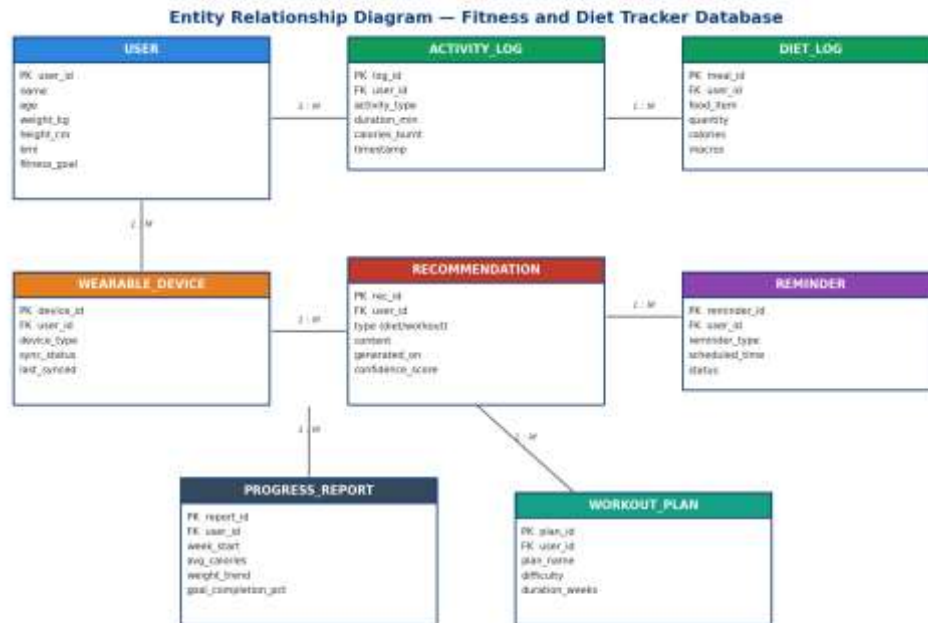


Figure 5: Entity Relationship Diagram of the Fitness and Diet Tracker Database

5.5 Deployment Diagram

Figure 6 shows the physical/cloud deployment topology, mapping software components onto Kubernetes nodes, managed database services, and third-party integrations.

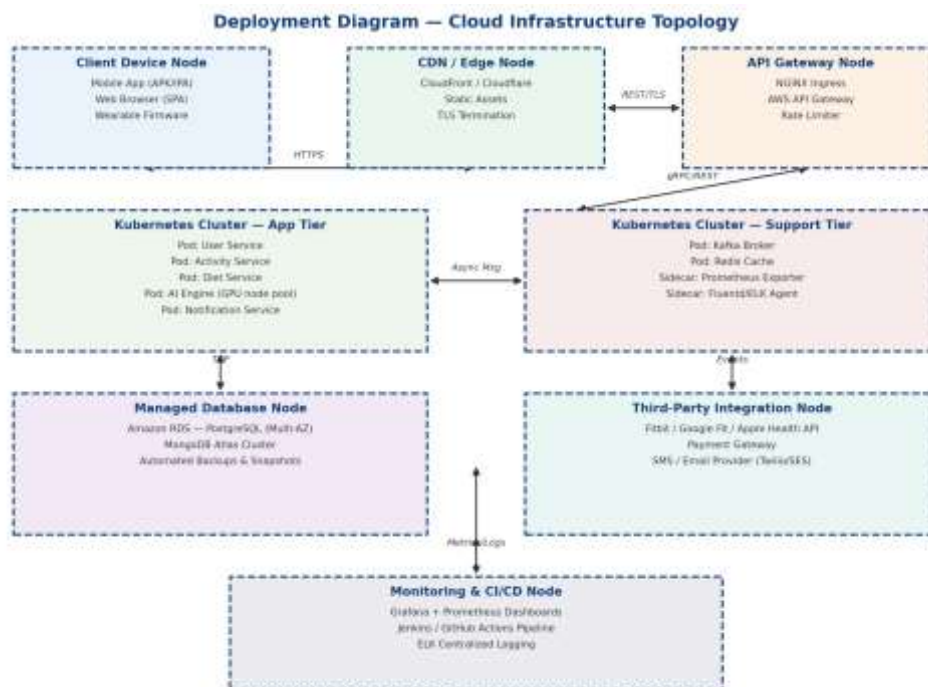


Figure 6: Deployment Diagram — Cloud Infrastructure Topology

5.6 UML Class Diagram

Figure 7 presents the core domain model as a UML class diagram, showing the principal classes, their key attributes and operations, and the associations that mirror the entity relationships defined in Section 5.4.

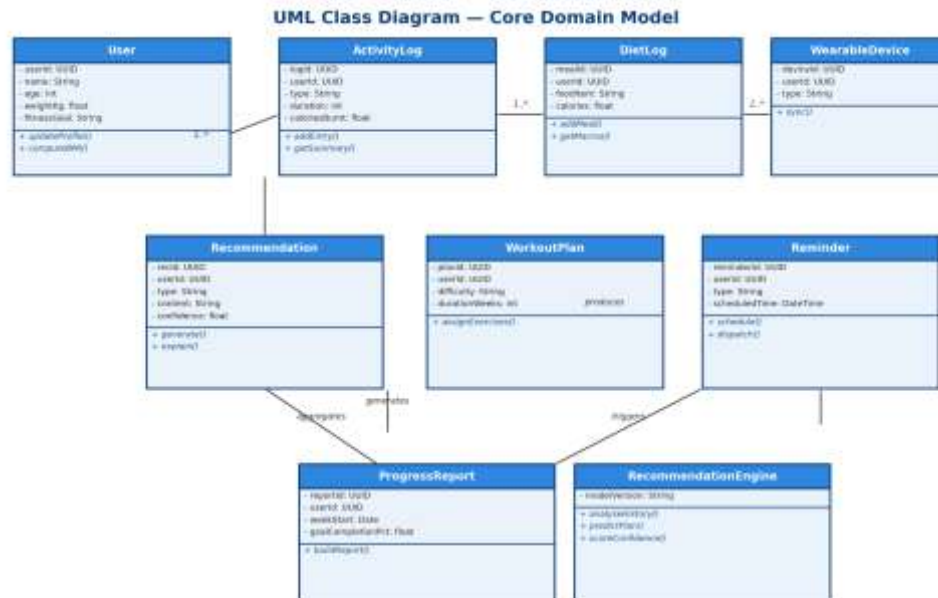


Figure 7: UML Class Diagram of the Core Domain Model

6. Discussion of Quality Attributes

Figure 8 summarises the architecture's projected achievement across the six quality attributes identified in Section 2.3, based on the design decisions detailed below

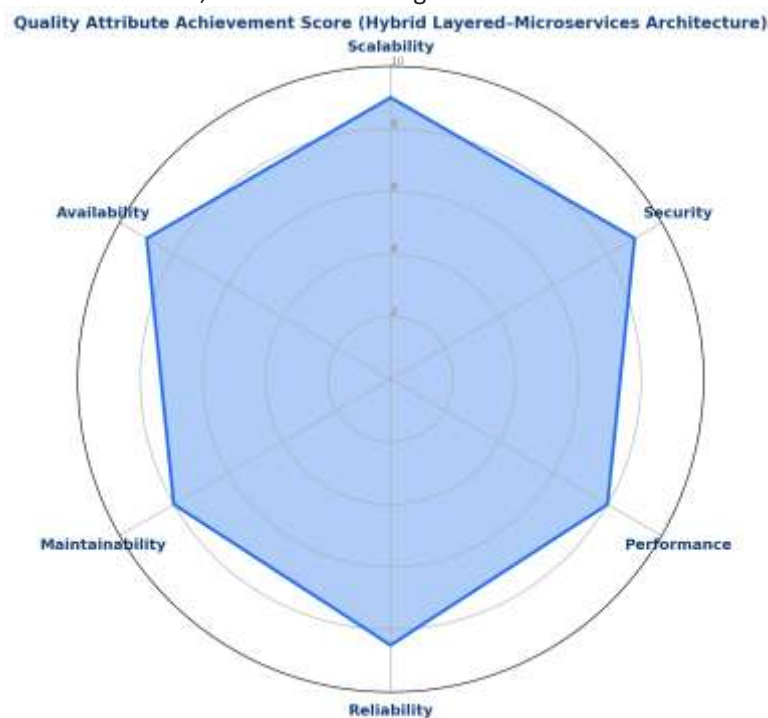


Figure 8: Quality Attribute Achievement Radar Chart

Attribute	How the Architecture Addresses It
Scalability	Each microservice scales independently on Kubernetes via Horizontal Pod Autoscaling; MongoDB shards handle high-volume wearable streams; Redis offloads read-heavy traffic.

Security	OAuth2/JWT-based authentication, TLS in transit, AES encryption at rest, RBAC-controlled access, and centralised audit logging protect sensitive health data end-to-end.
Performance	API Gateway load balancing, Redis caching of session and real-time metrics, and asynchronous message-driven processing keep dashboard and recommendation latency low.
Reliability	Kafka/RabbitMQ guarantees at-least-once delivery of events; database replication and automated backups prevent data loss during synchronisation or service failure.
Maintainability	Clear separation of microservices, the repository/DAO pattern, and a CI/CD pipeline allow independent development, testing, and deployment of each feature.
Availability	Multi-AZ managed databases, container orchestration with auto-restart, and redundant service replicas maintain the targeted 99.9% uptime.

7. Architectural Justification and Trade-offs

7.1 Rationale Behind Key Decisions

The decision to decompose the business logic into six microservices — rather than a single monolithic service — was driven directly by the differing scalability and reliability profiles identified in Section 2.3. The AI Recommendation Engine, in particular, is computationally intensive and benefits from GPU-backed node pools that would be wasteful to attach to the entire application. Placing it behind an asynchronous message broker also decouples recommendation generation from the user-facing request path, preventing slow AI inference from blocking meal-logging or dashboard operations.

7.2 Trade-offs Considered

- Operational complexity vs. independent scalability: Running six microservices increases deployment and monitoring overhead compared with a monolith, but this is mitigated by containerisation, centralised logging (ELK), and unified monitoring (Prometheus/Grafana).
- Eventual consistency vs. strict consistency: Asynchronous event-driven communication between the Diet Service and AI Engine introduces brief eventual-consistency windows, which is acceptable because recommendations are advisory rather than transactional.
- Polyglot persistence complexity vs. workload fit: Using PostgreSQL, MongoDB, and Redis together adds operational surface area but is justified because each data shape (relational, streaming, cache) is served far more efficiently than a single database engine could achieve.
- Development speed vs. long-term flexibility: A monolith would have been faster to build initially, but the hybrid approach was chosen to support the long-term goal of adding new health-tracking modules without re-architecting the whole system.

7.3 Support for Future Enhancement, Integration, and Maintainability

- New wearable providers can be integrated by extending only the Wearable Sync Service, without modifying other services.
- Additional AI models (e.g., injury-risk prediction, sleep-quality analysis) can be deployed as new microservices behind the same message broker.
- The API Gateway allows versioned APIs, enabling backward-compatible rollout of new mobile app features.
- Kubernetes-based deployment supports blue-green and canary releases, reducing risk during future upgrades.
- Clear component boundaries (Section 5.1) simplify onboarding of new developers and long-term maintainability of the codebase.

8. Technology Stack Summary

The following technology stack has been selected to realise the architecture described in Sections 3 to 5, balancing developer productivity, community support, and the performance/scalability requirements of a health-data platform.

Layer	Technology Choice	Reason
Frontend (Mobile)	React Native	Single codebase for Android and iOS reduces development effort.
Frontend (Web)	React + Tailwind CSS	Component reuse with the mobile app and rapid, consistent UI styling.
API Gateway	NGINX / AWS API Gateway	Mature, production-grade routing, TLS termination, and rate limiting.
Backend Services	Node.js (Express) / Java (Spring Boot)	High-throughput I/O for tracking services; strong typing for core business logic.
AI Recommendation Engine	Python, TensorFlow / Scikit-learn	Rich ML ecosystem for personalised recommendation models.
Messaging	Apache Kafka	High-throughput, durable event streaming for asynchronous workflows.
Relational Database	PostgreSQL	ACID-compliant storage for user, profile, and transactional data.
NoSQL Database	MongoDB	Flexible schema for high-volume activity and wearable telemetry.
Cache	Redis	Sub-millisecond access to session data and real-time metrics.
Containerisation	Docker + Kubernetes	Portable, auto-scaling deployment across cloud environments.
Monitoring	Prometheus + Grafana, ELK Stack	Real-time metrics visualisation and centralised log analysis.
CI/CD	GitHub Actions / Jenkins	Automated build, test, and deployment pipelines.

9. API Design Overview

The API Gateway exposes a set of versioned, RESTful endpoints consumed by the mobile and web clients. Representative endpoints for the core services are summarised below; all endpoints require a valid JWT bearer token issued by the Authentication component.

Endpoint	Method	Description
/api/v1/auth/register	POST	Registers a new user and creates an initial health profile.
/api/v1/auth/login	POST	Authenticates a user and issues an access/refresh token pair.
/api/v1/profile/{userId}	GET / PUT	Retrieves or updates a user's demographic and fitness-goal data.
/api/v1/activity/log	POST	Logs a new physical activity entry (type, duration, calories).
/api/v1/diet/log	POST	Logs a meal entry with quantity and computed nutritional values.

/api/v1/wearable/sync	POST	Accepts a batch of biometric readings pushed from a wearable device.
/api/v1/recommendations/{userId}	GET	Returns the latest AI-generated workout and diet recommendations.
/api/v1/dashboard/{userId}	GET	Returns aggregated progress data for dashboard visualisation.
/api/v1/reminders	POST / GET	Creates or lists scheduled reminders for meals, workouts, and hydration.

10. Risk Assessment and Mitigation

Risk	Impact	Mitigation Strategy
Wearable API rate limits or outages	Medium	Queue-and-retry via the message broker; graceful degradation to manual logging.
Health-data breach	High	End-to-end encryption, RBAC, regular penetration testing, and compliance audits.
AI recommendation inaccuracy	Medium	Continuous model evaluation, human-reviewable confidence scores, and user feedback loops.
Service overload during peak hours	Medium	Horizontal auto-scaling policies and Redis caching to absorb read-heavy traffic.
Data inconsistency from async events	Low	Idempotent event handlers and periodic reconciliation jobs.
Vendor lock-in with cloud provider	Low	Containerised, Kubernetes-based deployment portable across AWS, GCP, and Azure.

10.1 Data Privacy and Compliance Considerations

- All personally identifiable and health-related fields are encrypted at rest using AES-256 and in transit using TLS 1.3.
- Users can export or permanently delete their data on request, satisfying data-portability and right-to-erasure principles.
- Access to raw health data is restricted to the owning user and authorised roles via RBAC; nutritionist/admin views use aggregated, anonymised data wherever possible.
- Audit logs record every access to sensitive health records for traceability during compliance review.
- Third-party wearable integrations only request the minimum data scopes required (steps, heart rate, sleep) under the principle of least privilege.

11. Testing Strategy

- Unit Testing: Each microservice includes unit tests (Jest for Node.js, JUnit for Java) covering business-logic functions in isolation, targeting at least 80% code coverage.
- Integration Testing: Contract tests validate REST/gRPC interfaces between services, and message-broker integration tests confirm correct event publication and consumption.
- System Testing: End-to-end test suites (Cypress/Selenium) simulate complete user journeys such as registration, activity logging, and recommendation retrieval.
- Performance Testing: Load-testing tools (JMeter/k6) validate the API Gateway and AI Recommendation Engine under simulated peak concurrent users.
- Security Testing: Static analysis (SAST), dependency scanning, and periodic penetration testing verify compliance with the security requirements in Section 2.3.
- User Acceptance Testing: A pilot user group validates dashboard usability, recommendation relevance, and reminder effectiveness before full rollout.

12. Implementation Roadmap

Figure 9 presents a sprint-based delivery timeline for implementing the architecture described in this report, sequencing work so that foundational services are built before dependent AI and integration features.

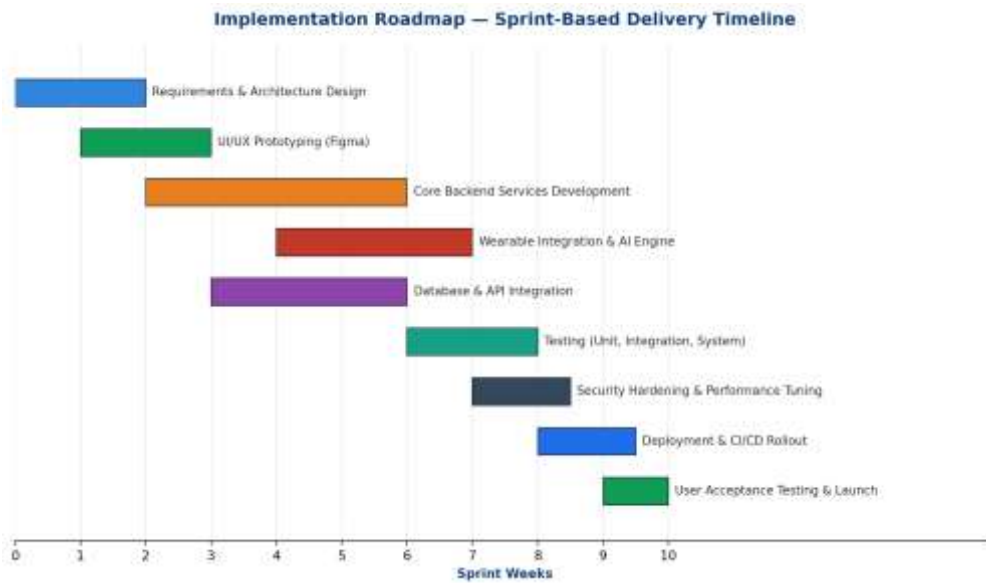


Figure 9: Implementation Roadmap — Sprint-Based Delivery Timeline

Architecture and requirements finalisation runs in parallel with early UI/UX prototyping to avoid idle time. Core backend services begin once the API contracts are frozen, while the AI Recommendation Engine and wearable integration start slightly later once the Diet and Activity services expose stable data models. Testing, security hardening, and deployment are scheduled as overlapping late-stage sprints to allow continuous feedback before the final user-acceptance and launch milestone.

12.1 Success Metrics and Acceptance Criteria

Metric	Target
API response time (p95)	Under 2 seconds for dashboard and recommendation endpoints.
System uptime	99.9% monthly availability, measured via Prometheus/Grafana dashboards.
Wearable sync success rate	At least 98% of sync attempts complete without manual retry.
Test coverage	Minimum 80% unit-test coverage across all microservices.
Security incidents	Zero critical vulnerabilities open beyond one sprint at release time.
User goal-completion rate	Measurable increase in weekly goal completion after 8 weeks of use.

13. Glossary of Terms and Abbreviations

Term	Definition
API	Application Programming Interface — a contract that allows software components to communicate.
BMI	Body Mass Index — a weight-to-height ratio used as a general health indicator.
CI/CD	Continuous Integration / Continuous Deployment — automated pipelines for building, testing, and releasing software.
DAO	Data Access Object — a design pattern that abstracts database operations behind a clean interface.
DFD	Data Flow Diagram — a visual representation of how data moves through a system.

ER Diagram	Entity Relationship Diagram — models database entities, attributes, and relationships.
JWT	JSON Web Token — a compact, signed token format used for stateless authentication.
MoSCoW	A prioritisation technique classifying features as Must, Should, Could, or Won't have.
RBAC	Role-Based Access Control — restricts system access based on user roles.
REST	Representational State Transfer — an architectural style for designing networked APIs.
TLS	Transport Layer Security — a protocol that encrypts data in transit.
UML	Unified Modeling Language — a standard notation for visualising software design.

14. Conclusion

This report analysed the requirements of the Fitness and Diet Tracker with Personalized Recommendations and selected a Hybrid Layered + Microservices Architecture as the most suitable design. The architecture, component, sequence, data-flow, entity-relationship, deployment, and class diagrams presented in Sections 4 and 5 collectively demonstrate a scalable, secure, and maintainable system capable of integrating wearable devices, generating AI-driven recommendations, and visualising user progress in real time. The quality-attribute analysis in Section 6 and the trade-off discussion in Section 7, together with the technology stack, API design, risk assessment, testing strategy, and implementation roadmap in Sections 8 to 12, confirm that the selected architecture satisfies the non-functional requirements identified during requirements analysis while remaining extensible for future enhancement.

References

- Bass, L., Clements, P., and Kazman, R. — Software Architecture in Practice, Addison-Wesley.
- Newman, S. — Building Microservices: Designing Fine-Grained Systems, O'Reilly Media.
- Fitbit Web API Documentation — developer.fitbit.com
- Google Fit REST API Documentation — developers.google.com/fit
- Apple HealthKit Developer Documentation — developer.apple.com/healthkit

Appendix A — Diagramming Tools and Methodology

All diagrams in this report (Figures 1–9) were produced programmatically using a Python-based diagramming toolchain rather than freehand drawing, ensuring precise alignment, consistent colour coding, and reproducibility. The architecture, component, sequence, data-flow, entity-relationship, deployment, class, and roadmap diagrams were generated using structured layout scripts, and the document itself was assembled using an automated document-generation toolchain to guarantee consistent formatting, heading styles, and table structures throughout.

Diagram	Tool / Method Used
Architecture & Component Diagrams (Figures 1–2)	Python (Matplotlib patches) — layered box-and-arrow layout.
Sequence Diagram (Figure 3)	Python (Matplotlib) — lifeline and message-arrow generation.
Data Flow & Entity Relationship Diagrams (Figures 4–5)	Python (Matplotlib) — process/entity notation with relationship lines.
Deployment Diagram (Figure 6)	Python (Matplotlib) — node-and-artifact topology layout.
UML Class Diagram (Figure 7)	Python (Matplotlib) — UML class-box notation with attributes/methods.
Quality Attribute Radar (Figure 8)	Python (Matplotlib polar projection) — quantitative attribute scoring.